



Hands-on No. : 01**Topic** : **Classes, Objects and Encapsulation****Date** : **10-12-25**

1.Account Management System

In modern financial organizations, every customer holds an account that must be accurately managed in terms of identification, balance, and transactions. To ensure reliability and data security, these details should be encapsulated within a structured programmatic model, allowing controlled access through public methods.

This models a **simplified banking system**, aligning with real-world account management scenarios used in enterprise applications.

System Overview

Class Name: Account

The Account class represents an individual customer's bank account. It securely encapsulates key account data and provides operations for transaction handling.

◆ Attributes (Private Data Members)

Attribute	Type	Description
id	String	Unique account identifier
name	String	Account holder's name
balance	int	Current account balance (default = 0)

◆ Constructors

Constructor	Description
Account(String id, String name)	Initializes a new account with default balance (0).

It is going to be hard but, hard does not mean impossible.

**Constructor****Description**

Account(String id, String name, int balance) Initializes a new account with a given balance.

◆ **Public Methods (Encapsulated Access)**

Method	Return Type	Description
getId()	String	Retrieves the account ID.
getName()	String	Retrieves the account holder's name.
getBalance()	int	Retrieves the current account balance.
credit(int amount)	int	Adds the specified amount to the balance and returns the updated balance.
debit(int amount)	int	Deducts the specified amount if sufficient balance is available; otherwise displays "Amount exceeded balance". Returns the updated balance.
transferTo(Account another, int amount)	int	Transfers the specified amount to another account if balance allows; otherwise displays "Amount exceeded balance". Returns the updated balance.
toString()	String	Returns a string representation of the Account Object

5. Functional Requirements

ID	Requirement	Description
FR1	Account Creation	System must allow creating new customer accounts with or without initial balance.
FR2	Credit Operation	System should allow adding funds to an account.
FR3	Debit Operation	System must validate and allow withdrawal if balance is sufficient.
FR4	Fund Transfer	System must transfer amount between two accounts safely.

It is going to be hard but, hard does not mean impossible.



ID	Requirement	Description
FR5	Account Display	System must display account information in formatted structure.

2. Employee Attendance Management

The organization aims to develop a **secure Employee Attendance Management System** to accurately record and monitor employees' **daily check-in and check-out activities**.

Each attendance record should capture essential **employee details** and corresponding **attendance timestamps**. The system must ensure that:

- Employees **cannot check in more than once per day**, and
- **Check-out actions** are allowed **only after a valid check-in**.

All attendance-related information shall be **securely encapsulated** to prevent unauthorized access or modification.

To achieve these objectives, the system will be designed using **two primary classes — Employee and Attendance — each responsible for distinct functional roles**.

System Overview

Class: Employee

Represents an employee in the organization responsible for marking attendance.

Member Type	Name	Description
Private Attributes	empId : int	Unique employee ID
	empName : String	Employee's full name
Public Methods	markCheckIn(Attendance attendance)	Marks check-in for the employee
	markCheckOut(Attendance attendance)	Marks check-out for the employee

Class: Attendance

Represents and manages the attendance record for each employee.

It is going to be hard but, hard does not mean impossible.



HANDS-ON

Member Type	Name	Description
Private Attributes	empId : int	Employee ID for linking attendance
	checkInTime : LocalTime	Time of check-in
	checkOutTime : LocalTime	Time of check-out
	isCheckedIn : boolean	Status flag for check-in
	isCheckedOut : boolean	Status flag for check-out
Public Methods	markCheckIn()	Records the current time as check-in
	markCheckOut()	Records the current time as check-out
	getAttendanceDetails()	Returns formatted attendance information

Functional Requirements

ID Requirement Description

- FR-1** The system shall allow an employee to check in only once per day.
- FR-2** The system shall allow an employee to check out only after checking in.
- FR-3** The system shall prevent multiple check-outs on the same day.
- FR-4** The system shall store employee ID, name, check-in, and check-out times.
- FR-5** All attendance details shall be securely encapsulated within the classes.
- FR-6** The system shall generate formatted attendance summaries for review.

3.Product Cart Management

An online retail organization seeks to design and implement a **robust and scalable Product Cart Management System** to efficiently manage the products customers add to their shopping carts.

Each product entry should include essential details such as **Product ID, Name, Price, and Quantity**. The system must ensure **data integrity** by restricting unauthorized modifications to product details while allowing users to safely update quantities when required.

It is going to be hard but, hard does not mean impossible.



HANDS-ON

The shopping cart should automatically **calculate and update the total cost** based on the items and their quantities, ensuring a smooth and accurate checkout experience.

The system adheres to **object-oriented design principles**, emphasizing **encapsulation, reusability, and maintainability** for long-term scalability and performance.

System Overview

Class 1: Product

Member Type	Name	Description
Private Attributes	productId : int	Unique identifier for the product.
	productName : String	Name or description of the product.
	price : double	Price per unit of the product.
	quantity : int	Quantity available or selected by the user.
Public Methods	getProductDetails()	Returns formatted details of the product.
	setQuantity(int newQty)	Safely updates product quantity while maintaining integrity.
	calculateTotal()	Returns the total cost based on price × quantity.

Class 2: Cart

Member Type	Name	Description
Private Attributes	products : List<Product>	List of all products currently in the cart.
Public Methods	addProduct(Product p)	Adds a new product to the cart.
	removeProduct(int productId)	Removes a specific product from the cart using its ID.
	calculateCartTotal()	Computes the total cost of all products in the cart.

It is going to be hard but, hard does not mean impossible.



HANDS-ON

Member Type **Name**

Description

displayCartItems()

Displays details of all products and the total cart value.

SmartCliff

It is going to be hard but, hard does not mean impossible.